

Git Commit Messages: Options and Approaches

chelseatroy.com/2018/09/23/git-commit-messages-options-and-approaches

View all posts by Chelsea

September 23, 2018

Reading Time: 12 minutes

I love git. I think it's a great tool for keeping my changes safe and integrating them with my team's changes.

After pairing with over 125 developers in my career so far, I have heard some opinions on commit messages. Most of them take an all-or-nothing approach: "Always do X in commit messages."

Rather than apply a single commit message pattern to all of my commits, I keep a mental directory of commit message patterns, and I try to apply the pattern that best suits the circumstances of each commit.

	COMMENT	DATE
○	CREATED MAIN LOOP & TIMING CONTROL	14 HOURS AGO
○	ENABLED CONFIG FILE PARSING	9 HOURS AGO
○	MISC BUGFIXES	5 HOURS AGO
○	CODE ADDITIONS/EDITS	4 HOURS AGO
○	MORE CODE	4 HOURS AGO
○	HERE HAVE CODE	4 HOURS AGO
○	AAAAA	3 HOURS AGO
○	ADKFJSLKDFJSDKLFJ	3 HOURS AGO
○	MY HANDS ARE TYPING WORDS	2 HOURS AGO
○	HAAAAAAAAAANDS	2 HOURS AGO

AS A PROJECT DRAGS ON, MY GIT COMMIT MESSAGES GET LESS AND LESS INFORMATIVE.

Let's go over some common styles, their advantages and limitations, and why we might choose one over another in various cases.

Commit Message Styles

Let's talk about a few styles of commit message, their strengths and limitations, and when each style might make sense in a project.

1. The Past Tense Style

Examples:

“Added caching to SomethingClass”

“Updated ApiCaller to use the new whales.com API”

Explanation: This message states what the developer did in this change set.

Advantages: The message answers the question “what feature does this add?” in just a few words. We value short commit messages for a few reasons:

- GitHub and some IDEs truncate commit messages in lists to 50 characters. If the first line of a commit message fits inside that limit, a developer scanning a commit list in one of those tools can quickly answer the question “is this the change set I am trying to read?”
- A short message indicates that one thing happened in this commit. We try to have each commit circumscribe just *one* change so that we could roll back that change without extricating it from other changes. If we have to list multiple changes, that’s a longer commit message.
- A short message indicates that the code doesn’t require further explanation: hopefully, it’s relatively self-explanatory. This is the same reason we want most code to stand on its own without comments, and it’s not always feasible. We’ll see another commit message style later to address these cases.

Limitations: Cannot share much context in this very brief message.

Great for: When the change is well-circumscribed, well documented, well-tested, and well-understood by the development team as a whole. They’re great commit messages to leave if another member of your team might use them for *commit tracing*. We talked about commit tracing [in this post](#), where we said the following about good commits for tracing:

***Clearly named, well-circumscribed commits:** I want my example app built and maintained with commits that concisely but completely explain what change they make (clearly named). This allows me to pull up `git log` and choose a commit that makes a change that sounds relevant to my interests. I also want each commit to contain all the code changes that contributed to the stated change and not to contain code changes that did not contribute to the stated change (well-circumscribed). I am learning: I want the advantage of understanding exactly how a given change was made without the distraction of distinguishing unrelated changes.*

2. The Present Tense Style

Examples:

“Adds caching to SomethingClass”

“Updates ApiCaller to use the new whales.com API”

Explanation: This one is the same as Past Tense Style *except* that, instead of describing what the developer *did*, it describes what this change set *does*. It centers the code as the subject of the commit message rather than the developer. Advantages, limitations, and use cases are the same as for the Past Tense Style.

3. The Imperative Style

Examples:

```
| "Add caching to SomethingClass"
```

```
| "Call the new whales.com API"
```

Explanation: This one is the same as Past and Present Tense Styles *except* that, instead of describing what the developer *did* or what the change set *does*, it commands the *computer* to do something (hence the word ‘imperative’). I’ve heard a few salt-and-pepper programmers who came up in the procedural programming days insist on this style. I use this style when it matters to my teammates that I use this style. Advantages, limitations, and use cases are the same as for the Past Tense Style.

4. The “Product Manager Legible” Style:

Examples:

```
| "When a user visits SomethingPage, they no longer have to wait a long time for the data to load."
```

```
| "User has access to the up to date information about whales from the new whales.com service."
```

Explanation: This message follows a similar format to the one commonly used in agile environments for writing user stories: “as a ____, when I ____, the app does ____, so that ____.” The point of this format is to force the team to think about *who* they’re developing for, *how* that person will use the app, *what* should happen when they do things in the app, and *why*. A relevant, well-scoped feature should have some answer to each of these. If it doesn’t, then we prioritize other stories over this story as we continue looking for missing information on this story: what the user wants, what the user expects, what we want to deliver to the user, or why it matters.

Advantages: This depends on where the developer is getting the commit message. If they’re copying and pasting it from a ticket/story, it’s easy to find the commit later that fulfills that story—which might be useful for backtracking later. If, instead, the developer is writing this commit message from scratch, then the developer *themselves* needs to understand what we want to accomplish and why. This is a good thing: developers have a better chance of building things people want to use if they understand their audience and the reasons for writing the things they’re tasked to write.

Limitations: If the team insists on this commit message style for everything, then we might be limited to one commit per story/ticket. This conflicts with the guideline of keeping commits *small* so they're digestible in code review and so we're recording our work with each incremental step forward. Additionally, when forced to do this, devs end up having to jam weird sentences into the format. For example: "When a developer writes code in the app, it will be in Java 8 and not Java 7, so that we can take advantage of new language features." This is entertaining, but I'd learn just as much from a shorter "Upgrades app from Java 7 to 8."

Great for: Small, user-facing features and close integrations between the tech team and the product team.

5. The "Developer Orientation" Style:

Example:

```
"Cache the SomethingPage call result. -The call takes 10 seconds to return, so users have to wait on SomethingPage. This way they only wait the first time they load the page. – Important: I chose to invalidate the cache after 12 hours. This is a guess. Too long and our data gets out of date. Too short and users wait around. We tracked data changes and found that the info only changes every day or so right now."
```

Explanation: This message provides context about how or why a committer made the choices in this change set.

Advantages: Sometimes it can be hard to convey in code why we did something. The commit message gives us another dedicated place to share our thoughts. The popular alternative to putting context in commit messages is to put it in comments on the code itself. I usually prefer to put the it in commit messages because that more strongly ties it to the time of writing.

Code comments can stick around after the code itself changes. It can then mislead team members with outdated context.

By contrast, commit messages are attached to (and typically displayed with) the date of the change:

```
8/31/18 } commit 00e0dceb861ef45607151a0b04e3bd8c47ea907f
8/31/18 } Author: Chelsea Troy
8/31/18 } Date: 8/31/18, 1:17 PM
9/3/18 }
9/3/18 }
9/10/18 } Organize notebooks in steps with a data preparation script to run in each model
```

When we `git annotate filename:line_number`, we see the SHA of the last commit in which this line changed. So if a line gets committed with context and then that line changes in a later commit, the new commit message overtakes the old one as the automatic context attached to this line.

```
9/18/18 Tr commit 52f01d34acea55e2e9445a63d09065f7a0de587a
9/10/18 Tr Author: Chelsea Troy
9/18/18 Tr Date: 9/10/18, 1:06 PM
9/10/18 Tr
9/18/18 Tr
9/18/18 Tr Add preprocessing functionality to Content class for splitting training and test data
```

Limitations: These commit messages can get long. That has two drawbacks. First, Github or your IDE may cut off the message. You may be able to mitigate this drawback by starting with a short title and then adding context afterward, so that even if it gets cut off the short title shows.

The second drawback is that these messages can make teammates nervous (*“oh gosh that’s a lot of message, what gnarly thing did we have to do here?”*) or they can make teammates stop reading messages (*“wow, a lot of these commit messages are so long. Too much noise to signal for me to read.”*) To mitigate this drawback, I commit like this sparingly—much the same way you might include code comments only sparingly.

Great for: When the change required integration with a complicated or surprising API, or when the change meant making a choice that might seem strange to another developer. I want to document my decisions where they don’t make sense at a first look.

6. The “I am tired and I want to push this” style:

“configuring the caching env var”

“Fix https for whales API”

“\$#!!!!!!!T”

Explanation: Developers dash this off when they just want to push the code, already.

Advantages: It gets the code out the door.

Limitations: It provides little or no information about the change set itself.

I see this type of commit in four cases.

Case #1: A developer is trying to fix code config for integration or deployment. They have to commit their changes and push them somewhere to see if it worked. This is by far the most common case.

I have been here. In these cases, I have started landing on this approach:

```
b3a25ef5b7a359d23df: "Attempt to connect to staging graph database on Heroku."
34b5e7s26a92ed61a8: "Attempt #2 at b3a25."
e845f6a2fe34719b93c: "Attempt #3 at b3a25."
```

That way I can get my tweaks out quickly while still referencing the original context.

Case #2: A developer is doing a hot fix and wants the change out the door *fast*.

I get this. We're under pressure and we have to get the thing out. In the heat of the moment, it's easy to get caught up in the rush and shove out code.

It's worth considering, though, that if we're doing a hot fix, then we're addressing an issue that [ticks *all* of our boxes for a risky change](#):

- effects are fairly catastrophic
- likely to happen (in fact, already happening)
- likely to go uncaught (having already made it to production uncaught)

If we had a risk map of our application, this area would be glowing red right now. As developers, it is our responsibility to mitigate risk with strategies to make sure things don't break and stay unbroken. That means that a *complete* hot fix will mean the inclusion of at least one regression test. If we think it's worth the time to scope, write, fail, and pass a test for this hot fix, then it's *certainly* worth the much smaller amount of time it takes to write an expressive commit message.

If that idealistic message doesn't convince you, maybe this will: the customer will be equally pissed if your fix takes fifteen minutes or if your fix takes fifteen minutes and twenty seconds. The time you took to write the commit message is not what lost you the customer on this hot fix. But *not* writing the commit message might lose you *another* customer if the team can't tell why this change was made and this same type of issue happens *again* later.

So please, dash off a commit message of some kind. Your future self will thank you.

Case #3: The developer is frustrated, angry, bored, or trying to be funny.

This is by far the *least* common reason I have seen for this style: most of the time with commit messages like this, it's either an integration/deployment thing or a hot fix.

On the rare occasion that these messages show up in the course of normal development, they're usually from developers who show other signs of immaturity as well. Some people learn faster than others to consider the impact of their commit messages, just as some people learn faster than others to consider the impact of their actions in general. I have written some in the past about [consequences](#), [ripple effects](#), and [building teamwork into employee evaluations](#), so I won't restate all that here. I'll say the short version: developers who regularly do this need some coaching. As I mentioned, these developers usually show several signs of immaturity that need addressing.

Case #4. The developer is brand new and doesn't know how to write a commit message yet.

These developers also need coaching, but it takes about two minutes and after that they write perfectly acceptable commit messages. Yay!

Conclusion

I think that we lose some nuance when we make inflexible prescriptive statements like “always write the commit message this way.”

Instead, we have a few different styles of commit messages, each of which serves us best in a different situation. We have short ones for explaining clear, well-circumscribed change sets. We have medium-length ones that transfer business context. We have longer ones for calling attention to our heterodox choices and tying those choices firmly to the time period in which they were made. Finally, we have dash-off commit messages for integration/deployment pushes and hot fixes. Having all of these in your tool belt gives you more options when the time comes to share your code with the world—or at least with your team.

If you liked this post and you sometimes think about commit messages, you might like:

[Build Graceful Processes](#) (An ongoing series on software design that’s going in some directions you might not expect)

[Edge Free Programming by Michael Feathers](#) (the talk, not the book)

[Diagramming Data](#) (the super-exciting series on data integrity that you’ve always wanted!)